

浙江大学

本科实验报告

课程名称：数字系统设计实验

实验名称：音乐播放器设计实验

姓 名：

学 院：信息与工程学院

专 业：信息工程

学 号：

指导教师：屈民军、唐奕

目录

1	实验目的	1
1.1	Lab 11: 异步输入同步器和开关防颤动电路	1
1.2	Lab 15: 直接数字频率合成技术 (DDS) 的设计与实现	1
1.3	Lab 19: 音乐播放器综合实验	1
2	实验任务与要求	2
2.1	Lab 11 异步输入同步器和开关防颤动电路	2
2.2	Lab 15 直接数字频率合成技术 (DDS) 的设计与实现	2
2.3	Lab 19 音乐播放器综合实验	2
3	实验原理	2
3.1	系统总体原理	2
3.2	Lab 11: 异步输入同步和防颤动原理	3
3.2.1	异步输入与亚稳态	3
3.2.2	按键防颤动	3
3.2.3	脉冲宽度变换	3
3.3	Lab 15: DDS 正弦信号发生器原理	4
3.3.1	DDS 基本结构	4
3.3.2	四分之一波形 ROM 优化	4
3.4	Lab 19: 音乐播放器原理	5
3.4.1	音符编码与乐曲存储	5
3.4.2	主控制器 mcu	5
3.4.3	乐曲读取模块 song_reader	5
3.4.4	音符播放模块 note_player	6
3.4.5	节拍基准和音频接口	6
4	总体设计方案与模块划分	7
4.1	总体设计思路	7
4.2	顶层模块接口设计	7
4.3	模块层次划分	8
5	主要仪器设备与软件环境	8
6	实验步骤与过程	9
6.1	Lab 11: 按键处理模块设计与验证	9
6.2	Lab 15: DDS 模块设计与仿真	9
6.3	Lab 19: 音乐播放器系统集成	9

7 Verilog HDL 代码设计说明	9
7.1 按键处理模块 button_unit 设计与代码	9
7.2 主控制器 mcu 设计与代码	11
7.3 乐曲读取模块 song_reader 设计与代码	13
7.4 音符播放模块 note_player 设计与代码	16
7.5 DDS 模块 dds 设计与代码	18
7.6 采样同步与节拍产生模块设计与代码	21
7.7 顶层模块 music_player 设计与代码	22
7.8 复位与模块间握手机制	24
8 仿真调试与数据记录	24
8.1 Lab 11 按键处理仿真	24
8.2 DDS 模块仿真分析	25
8.3 MCU 主控制器模块仿真分析	27
8.4 song_reader 乐曲读取模块仿真分析	27
8.5 note_player 音符播放模块仿真分析	28
8.6 music_player 顶层模块仿真分析	30
9 下载验证与实验结果分析	31
9.1 开发板验证步骤	31
9.2 实验结果记录	32
9.3 结果分析	32
10 实验心得	32
11 思考题	33

浙江大学 实验报告

专业： 信息工程
姓名： _____
学号： _____
日期： 2026.5.20
地点： 东四 223

课程名称： 数字系统设计实验 指导老师： 屈民军、唐奕 成绩： _____
实验名称： 音乐播放器设计实验 实验类型： 综合设计性实验 同组学生姓名： 无

一、实验目的

本实验以音乐播放器为最终系统目标，将异步输入同步与开关防颤动电路、直接数字频率合成（DDS）模块以及音乐播放控制系统组合在同一数字系统中完成。通过本实验，掌握从底层功能模块到顶层系统的“自顶而下”设计方法，并理解各子模块之间的控制、数据和时钟关系。

1.1 Lab 11：异步输入同步器和开关防颤动电路

1. 掌握减少亚稳态的方法，理解亚稳态给同步数字系统带来的可靠性问题。
2. 掌握开关、按键机械颤动的消除方法。
3. 初步了解控制器在按键处理电路中的设计方法。
4. 为音乐播放器中的 `play/pause_button`、`next_button` 等人工按键输入提供稳定、单周期的控制脉冲。

1.2 Lab 15：直接数字频率合成技术（DDS）的设计与实现

1. 了解在数字域产生正弦信号的方法。
2. 掌握 DDS 技术的基本原理及其在频率可控正弦波产生中的应用。
3. 掌握用 ModelSim 观察波形并验证 DDS 输出频率变化的方法。
4. 为音乐播放器中的音符播放模块提供可由相位增量 K 控制频率的正弦波样品序列。

1.3 Lab 19：音乐播放器综合实验

1. 掌握音符产生的方法，理解 DDS 技术在乐音合成中的应用。
2. 了解音频编解码接口及 ADAU1761 音频芯片在数字音频输出中的作用。
3. 掌握复杂数字系统的模块划分、层次化设计和顶层集成方法。
4. 通过按键控制、乐曲读取、音符定时、DDS 采样输出和音频接口连接，实现可播放多首乐曲的音乐播放器。

二、实验任务与要求

2.1 Lab 11 异步输入同步器和开关防颤动电路

1. 设计一个按键处理模块，要求每按下一次按键，模块输出一个正脉冲信号。
2. 输出脉冲宽度为一个 `clk` 时钟周期，便于后级控制器准确识别一次按键操作。
3. 将按键处理模块接入测试电路，在开发板上验证每按下一次按键，LED 指示灯状态改变一次。

2.2 Lab 15 直接数字频率合成技术 (DDS) 的设计与实现

设计并仿真一个 DDS 正弦信号序列发生器，主要指标如下：

1. 采样频率 $f_c = 48\text{ kHz}$ 。
2. 正弦信号频率范围为 $20\text{ Hz} \sim 20\text{ kHz}$ 。
3. 正弦信号序列宽度为 16 位，其中包含 1 位符号位。
4. DDS 模块应能根据相位增量 K 改变输出正弦波频率，并输出 `new_sample_ready` 采样有效脉冲。

2.3 Lab 19 音乐播放器综合实验

1. 设计一个音乐播放器，能够播放四首乐曲。
2. 设置 `play/pause_button`、`next_button`、`reset` 三个按键；按 `play/pause_button` 时，音乐在播放与暂停之间切换；按 `next_button` 时，播放器切换到下一首乐曲。
3. 使用 LED0 指示播放状态：播放时点亮，暂停时熄灭。
4. 使用 LED2 和 LED3 指示当前乐曲序号。
5. 完成主要模块的 Verilog HDL 设计、ModelSim 功能仿真、Vivado 综合实现和开发板下载验证。

三、实验原理

3.1 系统总体原理

音乐播放器系统可分为时钟管理模块、按键处理模块、主控制器 `mcu`、乐曲读取模块 `song_reader`、音符播放模块 `note_player`、DDS 模块、节拍基准产生器、同步化电路和音频编解码接口模块。系统中按键输入经 Lab 11 的防颤动和单脉冲处理后送入主控制器；主控制器产生播放控制信号和乐曲序号；乐曲读取模块依次输出音符与音长；音符播放模块根据音符查询相位增量并调用 Lab 15 的 DDS 模块产生正弦样品；音频接口模块将样品转换为串行音频数据并送入 ADAU1761。

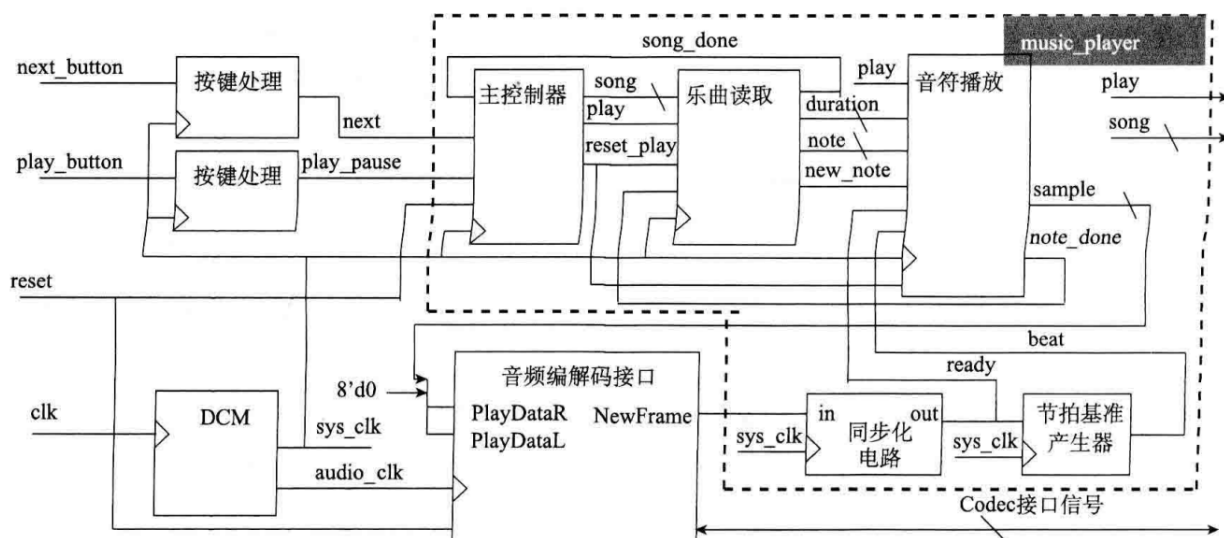


图 1: 音乐播放器顶层模块设计框图

3.2 Lab 11: 异步输入同步和防颤动原理

3.2.1 异步输入与亚稳态

外部按键、开关输入与系统时钟 `clk` 之间通常没有固定相位关系，属于异步输入。若异步信号在触发器建立时间或保持时间窗口内变化，触发器输出可能短时间停留在不确定状态，即亚稳态。亚稳态可能导致后续逻辑误判，因此需要使用同步器降低亚稳态传播概率。常用方案是将异步输入串接两级或多级 D 触发器，使信号逐级同步到系统时钟域。

3.2.2 按键防颤动

机械按键在按下和释放瞬间会产生数毫秒量级的抖动，表现为逻辑电平在 0 和 1 之间快速、不规则跳变。若直接送入控制器，可能被识别为多次按键。本实验采用“等待稳定一定时确认—状态更新”的思路：当检测到输入变化时启动约 10 ms 的定时器，定时结束后若电平保持稳定，则确认按键状态变化；否则继续等待。

在 100 MHz 系统时钟下，一个时钟周期为 10 ns。可先用分频器产生 1 kHz 定时基准，再用计数器累计 10 个基准脉冲形成约 10 ms 的防颤动时间窗口。

3.2.3 脉冲宽度变换

控制器通常只需要识别“按下一次”这一事件，而不需要持续高电平。因此，防颤动后的稳定按键信号还需要转换为宽度为一个系统时钟周期的脉冲。该脉冲送入 `mcu` 后，能够避免一次长按被重复识别。

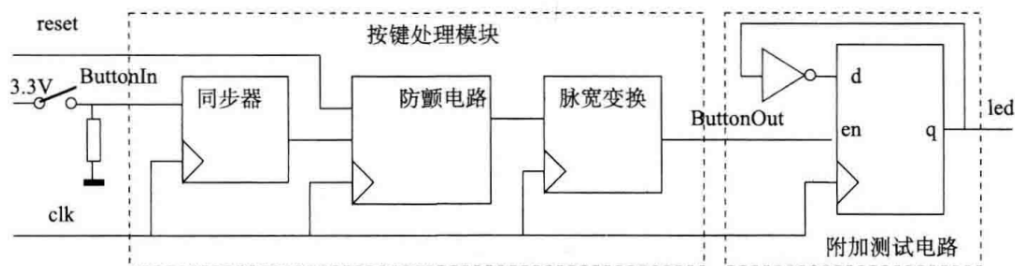


图 2: 按键处理模块设计框图

3.3 Lab 15: DDS 正弦信号发生器原理

3.3.1 DDS 基本结构

DDS (Direct Digital Synthesis) 通过相位累加器、正弦查找表和输出寄存器在数字域产生正弦波。每个采样时钟到来时，相位累加器加上相位增量 K ，累加结果的高位作为正弦 ROM 地址，从而依次读出正弦样品。改变 K 即改变查表步长，因此可改变输出正弦波频率。

正弦查找表样品可按式(3.1)生成：

$$S(i) = (2^{n-1} - 1) \sin\left(\frac{2\pi i}{2^m}\right), \quad i = 0, 1, \dots, 2^m - 1 \quad (3.1)$$

其中， m 为完整周期正弦表地址位数， n 为 ROM 数据位宽，本实验中输出样品位宽为 16 位。

若相位累加器有效位数为 M ，采样频率为 f_c ，输出频率 f_o 与相位增量 K 的关系可表示为：

$$f_o = \frac{K \cdot f_c}{2^M} \quad (3.2)$$

本实验根据最低频率分辨率要求采用 12 位整数地址，并增加 10 位小数位以提高频率精度，因此相位累加器位宽可取 $M = 22$ ，高 12 位用于表示完整周期正弦样品地址。

3.3.2 四分之一波形 ROM 优化

由于正弦波具有对称性，完整周期可分为四个象限。系统不必存储全部 2^{12} 个样品，而只需存储第 0 象限的 $2^{10} = 1024$ 个 16 位样品，再通过地址映射和数据取反/取补恢复完整波形。这样可显著降低 ROM 容量，并保持输出波形的连续性。

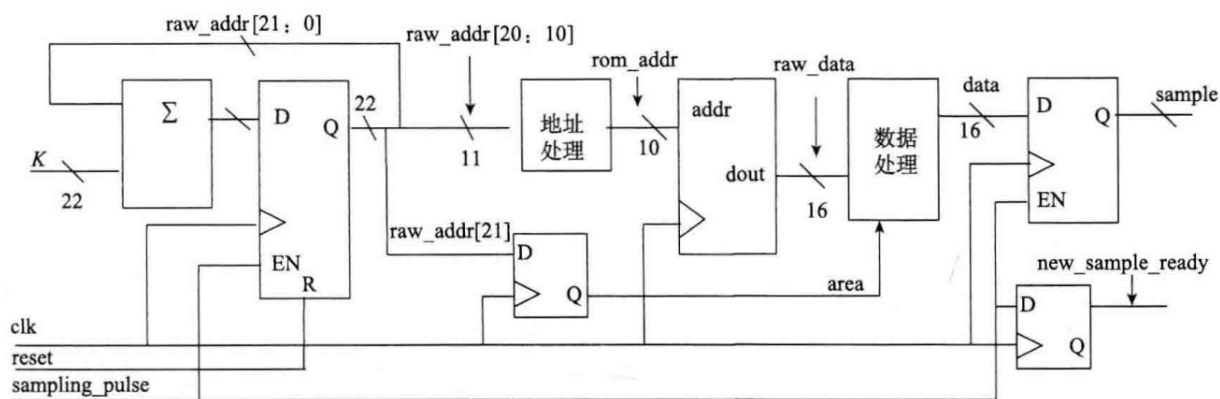


图 3: DDS 模块设计框图

3.4 Lab 19: 音乐播放器原理

3.4.1 音符编码与乐曲存储

音乐由音符及其持续时间组成。系统使用 `song_rom` 存储乐曲，每个存储单元保存 `{note,duration}` 信息，其中 `note` 表示音符编号，`duration` 表示音符持续时间。每首乐曲最多由 32 个音符组成，系统通过乐曲序号选择不同乐曲，通过低位地址依次读取同一首乐曲中的音符。

3.4.2 主控制器 mcu

主控制器负责响应 `reset`、`play/pause`、`next` 和 `song_done` 等信号，输出播放状态 `play`、乐曲序号 `song` 和重新开始播放脉冲 `reset_play`。mcu 可设计为有限状态机，典型状态包括 `RESET`、`PAUSE`、`NEXT` 和 `PLAY`。

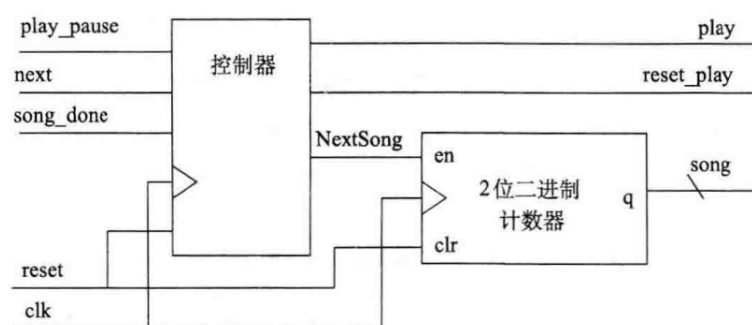


图 4: mcu 模块设计框图

3.4.3 乐曲读取模块 song_reader

`song_reader` 根据主控制器给出的 `play` 和 `song` 信号，从 `song_rom` 中读取当前乐曲的音符信息。当 `note_player` 发出 `note_done` 后，`song_reader` 读取下一个音符并产生 `new_note` 脉冲；当检测到乐曲结束标志时，产生 `song_done` 反馈给 mcu。

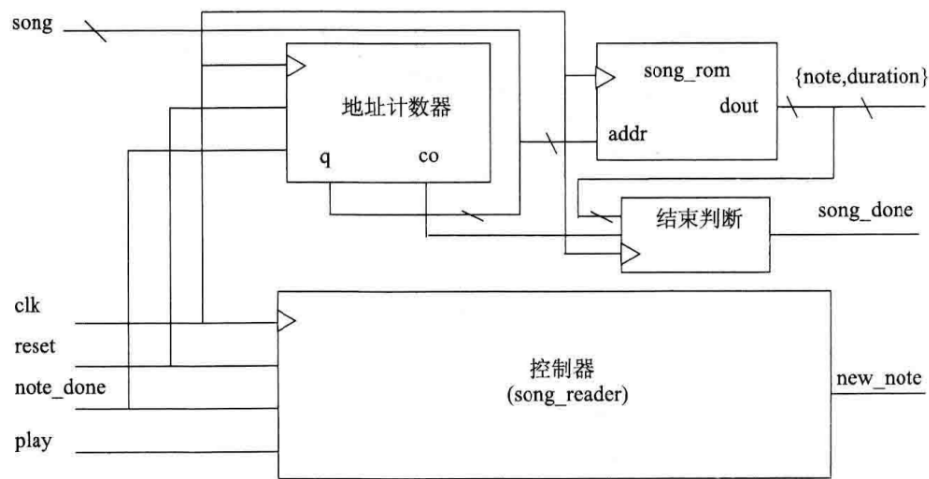


图 5: song_reader 模块设计框图

3.4.4 音符播放模块 `note_player`

`note_player` 是音乐播放器的核心模块，主要功能包括：接收 `song_reader` 输出的 `note` 和 `duration`；通过 `FreqROM` 查询该音符对应的 DDS 相位增量 K ；在音符持续时间内以 48 kHz 采样率调用 DDS 模块产生正弦样品；当音符时间结束后产生 `note_done`，请求下一个音符。

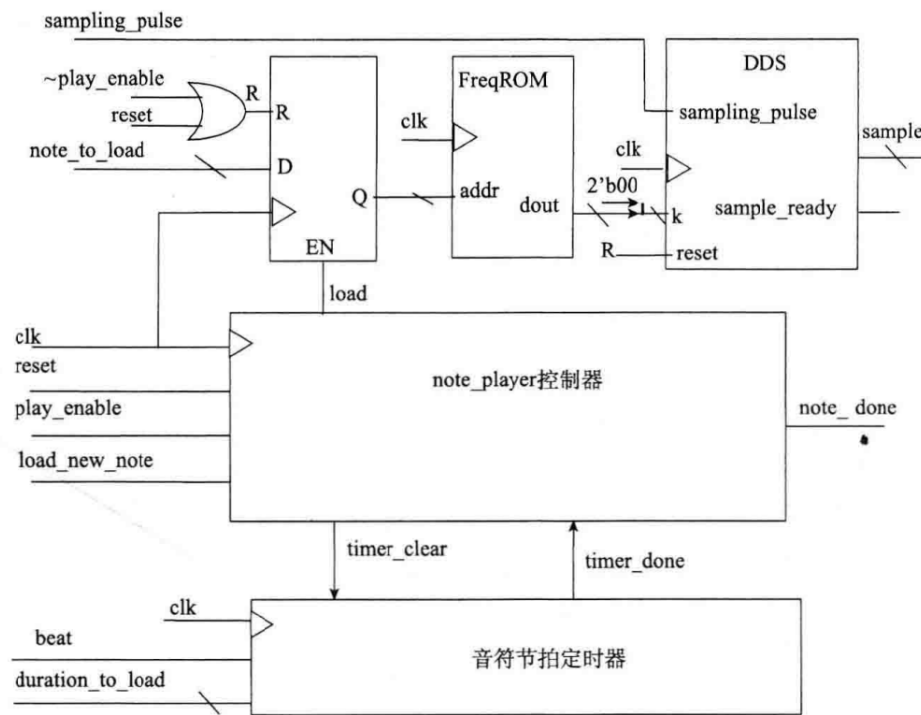


图 6: note_player 模块设计框图

3.4.5 节拍基准和音频接口

节拍基准产生器用于产生音符定时所需的 `beat` 脉冲。由于音频接口的 `ready` 信号频率为 48 kHz，若以 48 Hz 作为节拍基准，则可使用分频比为 1000 的计数器产生 `beat`。DDS 输出的 16 位正弦样品在送入音频编

解码接口时，需要扩展为音频接口所需的数据格式，例如在低位补 0 后送入串行音频模块。

四、总体设计方案与模块划分

4.1 总体设计思路

本实验最终实现的音乐播放器采用层次化设计方法，将 Lab 11 的按键同步与防颤动模块、Lab 15 的 DDS 正弦波发生模块以及 Lab 19 的乐曲控制模块组合为一个完整的数字音频系统。系统的基本工作流程如下：首先，外部按键经过按键处理单元转换为系统时钟域内的单周期控制脉冲；随后主控制器根据播放/暂停、下一首以及歌曲播放结束信号决定系统处于播放、暂停、切歌或复位状态；乐曲读取模块根据当前乐曲编号从乐曲 ROM 中依次取出音符编码和音长；音符播放模块根据音符编码查表得到 DDS 频率控制字，并在音符持续时间内不断输出对应频率的正弦采样值；最后，采样值送往音频编解码接口，由外部音频芯片完成数模转换和功率驱动。

从数据流角度看，系统的数据主线为

$$\text{song} \rightarrow \text{song_rom} \rightarrow \{\text{note}, \text{duration}\} \rightarrow \text{frequency_rom} \rightarrow K \rightarrow \text{dds} \rightarrow \text{sample}.$$

其中 **song** 决定当前播放哪一首乐曲，**note** 决定当前音符的音高，**duration** 决定当前音符的持续时间，DDS 频率控制字 **K** 决定正弦采样序列的频率。控制流则主要由 **play**、**reset_play**、**new_note**、**note_done** 和 **song_done** 等握手信号构成。这样既保证了各模块功能边界清晰，也便于分别进行模块级仿真和顶层联调。

4.2 顶层模块接口设计

本次代码中的播放器核心顶层模块为 **music_player**。该模块不直接接入原始机械按键，而是接收已经经过按键处理后的 **play_pause** 和 **next** 单周期脉冲；同时接收音频接口给出的 **NewFrame** 帧同步信号，并通过同步化电路产生系统时钟域内的采样请求信号。其主要接口如表1所示。

端口名	方向	功能说明
clk	Input	系统时钟信号，供主控制器、乐曲读取、音符播放和节拍分频等模块使用。
reset	Input	异步高电平复位信号，用于使播放器回到初始状态。
play_pause	Input	播放/暂停控制脉冲，由按键处理模块输出，高电平宽度为一个时钟周期。
next	Input	下一首控制脉冲，由按键处理模块输出，高电平宽度为一个时钟周期。
NewFrame	Input	音频编解码接口输出的帧同步信号，表示音频接口需要新的采样数据。
sample[15:0]	Output	DDS 产生的 16 位有符号正弦采样值，送往音频编解码接口。
play	Output	当前播放状态，可接 LED0 作为播放指示信号。
song[1:0]	Output	当前乐曲序号，可接 LED2、LED3 作为乐曲编号指示信号。

表 1: music_player 顶层模块接口

在板级顶层设计中,原始 **play_button** 和 **next_button** 应先分别接入 **button_unit**,得到稳定的 **play_pause** 和 **next** 脉冲后再送入 **music_player**。**music_player** 输出的 **play** 和 **song[1:0]** 则可直接用于 LED 指示，其中 **play** 对应播放状态，**song[1:0]** 对应当前曲目编号。

4.3 模块层次划分

根据上传的 Verilog HDL 文件，整个系统可以划分为按键处理层、播放控制层、乐曲读取层、音符播放层、DDS 波形产生层和采样同步层。各主要源文件与模块功能如表2所示。

源文件	主要模块	功能说明
button_unit.v	button_unit	对原始按键输入进行同步、消抖和上升沿检测，输出单周期按键脉冲。
music_player.v	music_player	播放器核心顶层，例化同步化电路、节拍分频器、主控制器、乐曲读取模块和音符播放模块。
sync_pulse.v	sync_pulse	将音频接口的 NewFrame 异步输入同步到系统时钟域，并产生单周期 ready 脉冲。
divider.v	divider	可参数化分频器，计数到设定值后输出一个时钟周期宽度的 beat 节拍脉冲。
mcu.v	mcu	主控制状态机，产生 play 、 reset_play 和当前乐曲编号 song 。
song_reader.v	song_reader	根据当前乐曲编号和音符地址读取 song_rom ，输出 note 、 duration 和 new_note 。
note_player.v	note_player	根据音符编码查找频率控制字，控制音符定时，并调用 DDS 产生正弦采样。
dds.v	dds	直接数字频率合成模块，根据频率控制字 K 输出 16 位正弦波采样值。

表 2: 主要 Verilog 源文件及其模块功能

系统的层次关系可概括为：**button_unit** 位于播放器核心模块外部，负责将机械按键转换为可靠控制脉冲；**music_player** 是核心顶层模块，内部包含 **sync_pulse**、**divider**、**mcu**、**song_reader** 和 **note_player**；**note_player** 内部进一步例化 **frequency_rom**、**note_timer**、**note_reg**、**note_player_controller** 和 **dds**；**dds** 内部再由相位累加器、地址处理、**sine_rom** 和数据符号处理电路组成。

五、主要仪器设备与软件环境

1. 计算机一台，安装 Vivado 与 ModelSim SE。
2. Nexys Video Artix-7 FPGA 开发板。
3. ADAU1761 音频编解码器及其音频输出接口。
4. 扬声器或耳机，用于验证音乐播放效果。
5. Verilog HDL 源代码编辑环境、约束文件和实验提供的音频接口文件。

六、实验步骤与过程

6.1 Lab 11: 按键处理模块设计与验证

1. 编写同步器模块，将异步按键输入同步到系统时钟域。
2. 编写 10 ms 防颤动定时器和状态控制器，实现稳定按键电平输出。
3. 编写脉冲宽度变换电路，将稳定按键事件转换为单时钟周期脉冲。
4. 编写测试平台，在 ModelSim 中观察输入抖动、稳定输出和单周期脉冲之间的关系。

6.2 Lab 15: DDS 模块设计与仿真

1. 根据式(3.1)生成四分之一周期 `sine_rom` 数据。
2. 设计 22 位相位累加器，在 `sampling_pulse` 有效时更新相位地址。
3. 根据相位高位判断当前象限，完成 ROM 地址映射和数据符号处理。
4. 使用不同相位增量 K 进行仿真，验证输出正弦样品频率随 K 增大而增大。
5. 将 DDS 模块封装，供 `note_player` 调用。

6.3 Lab 19: 音乐播放器系统集成

1. 搭建 `mcu` 有限状态机，完成播放/暂停、下一首和复位控制。
2. 搭建 `song_reader`，实现乐曲选择、音符顺序读取和歌曲结束判断。
3. 搭建 `note_player`，实现音符载入、音长计时、DDS 相位增量查询和样品输出。
4. 设计 `beat_generator`，由 48 kHz 采样有效信号分频得到 48 Hz 节拍。
5. 完成音频接口连接，将 DDS 输出样品送入音频编解码模块。
6. 进行各子模块仿真和顶层联调，最后下载到 FPGA 开发板验证实际播放效果。

七、Verilog HDL 代码设计说明

7.1 按键处理模块 `button_unit` 设计与代码

按键处理模块采用 `button_unit` 实现，其输入为系统时钟 `clk`、复位 `reset` 和原始按键输入 `ButtonIn`，输出为 `ButtonOut`。由于机械按键输入既可能与系统时钟异步，又存在按下或释放过程中的颤动，因此该模块按以下三级结构设计：

1. **同步器**：通过 `sync_ff` 将异步的 `ButtonIn` 同步到 `clk` 时钟域，降低亚稳态向后级电路传播的概率。
2. **消抖滤波器**：通过 `debounce_filter` 对同步后的按键信号进行稳定性判断，滤除机械触点抖动。模块带有参数 `sim`，便于在仿真时缩短消抖等待时间，在硬件下载时使用正常消抖时间。
3. **上升沿检测**：用寄存器保存上一拍消抖输出，当检测到当前消抖信号为 1 且上一拍为 0 时，输出一个时钟周期宽度的 `ButtonOut` 脉冲。

该设计保证了每次有效按键动作只会触发一次控制事件。因此，在播放控制中，`play_pause` 不会因为按键抖动而在播放和暂停之间连续切换，`next` 也不会因为一次按键而跳过多首乐曲。

Listing 1: button_unit 模块源代码

```
1 module button_unit #(
2     parameter sim = 0          // 消抖模块参数: 通常 0 表示硬件模式, 1 表示仿真模式
3 ) (
4     input wire clk,            // 系统时钟
5     input wire reset,          // 异步高电平复位
6     input wire ButtonIn,        // 原始按键输入, 可能存在抖动且与 clk 异步
7     output wire ButtonOut       // 消抖后的上升沿单周期脉冲
8 );
9
10 wire synced;                  // 同步后的按键信号
11 wire debounced;              // 消抖后的稳定按键信号
12
13 // 将异步按键输入同步到 clk 时钟域。
14 sync_ff u_sync (
15     .clk      (clk),
16     .reset     (reset),
17     .async_in  (ButtonIn),
18     .sync_out  (synced)
19 );
20
21 // 消抖滤波器: sim 参数由顶层传入, 便于仿真时缩短消抖时间。
22 debounce_filter #(
23     .sim(sim)
24 ) u_debounce (
25     .clk      (clk),
26     .reset     (reset),
27     .signal_in (synced),
28     .signal_out (debounced)
29 );
30
31 // 保存上一拍消抖信号, 用于生成上升沿单周期脉冲。
32 reg debounced_reg;
33
34 always @(posedge clk or posedge reset) begin
35     if (reset) begin
36         debounced_reg <= 1'b0;
37     end else begin
38         debounced_reg <= debounced;
39     end
40 end
41
42 // 上升沿检测: 当前为 1 且上一拍为 0。
43 assign ButtonOut = debounced & ~debounced_reg;
44
45 endmodule
```

7.2 主控制器 mcu 设计与代码

mcu 是系统播放行为的核心控制模块。其输入包括 play_pause、next 和 song_done，输出包括播放状态 play、乐曲编号 song[1:0] 和播放链路复位信号 reset_play。代码中将主控制器设计为四状态有限状态机，状态含义如下：

状态	状态含义	主要输出
RESET	系统复位或歌曲播放结束后的初始状态	play=0, reset_play=1, 清空后级播放链路。
PAUSE	暂停状态，等待播放或切歌命令	play=0, 保持当前歌曲编号。
PLAY	正常播放状态	play=1, 允许 song_reader 和 note_player 工作。
NEXT	切换下一首的瞬间状态	next_song=1, reset_play=1, 歌曲编号加一并复位播放链路。

表 3: mcu 状态机状态说明

状态转移逻辑为：复位后进入 RESET，随后自动进入 PAUSE；在 PAUSE 状态下若检测到 play_pause 脉冲，则进入 PLAY；在 PLAY 状态下再次检测到 play_pause，则回到 PAUSE；无论在暂停还是播放状态，只要检测到 next，均进入 NEXT 并使歌曲序号计数器加一；当 song_done 有效时，说明当前歌曲播放结束，控制器回到 RESET，从而复位音符读取和播放链路。

Listing 2: mcu 模块源代码

```
1 `timescale 1ns / 1ps
2
3 module mcu (
4     input wire      clk,           // 时钟信号
5     input wire      reset,         // 异步复位，高有效
6     input wire      play_pause,    // 播放/暂停切换脉冲
7     input wire      next,          // 下一首切换脉冲
8     input wire      song_done,     // 当前歌曲播放完成
9     output reg      play,          // 播放使能状态
10    output wire [1:0] song,         // 当前歌曲序号
11    output reg      reset_play     // 播放链路复位信号
12 );
13
14 // 状态编码
15 localparam RESET = 2'b00,
16             PAUSE = 2'b01,
17             PLAY  = 2'b10,
18             NEXT  = 2'b11;
19
20 reg [1:0] current_state;
21 reg [1:0] next_state;
22
```

```
23 reg next_song; // 歌曲序号计数使能
24
25 // 状态寄存器
26 always @(posedge clk or posedge reset) begin
27     if (reset)
28         current_state <= RESET;
29     else
30         current_state <= next_state;
31 end
32
33 // 次态逻辑
34 always @(*) begin
35     case (current_state)
36         RESET: begin
37             next_state = PAUSE;
38         end
39
40         PAUSE: begin
41             if (play_pause)
42                 next_state = PLAY;
43             else if (next)
44                 next_state = NEXT;
45             else
46                 next_state = PAUSE;
47         end
48
49         PLAY: begin
50             if (play_pause)
51                 next_state = PAUSE;
52             else if (next)
53                 next_state = NEXT;
54             else if (song_done)
55                 next_state = RESET;
56             else
57                 next_state = PLAY;
58         end
59
60         NEXT: begin
61             next_state = PLAY;
62         end
63
64         default: begin
65             next_state = RESET;
66         end
67     endcase
68 end
69
70 // 输出逻辑
71 always @(*) begin
```

```
72     case (current_state)
73         RESET: begin
74             play      = 1'b0;
75             next_song = 1'b0;
76             reset_play = 1'b1;
77         end
78
79         PAUSE: begin
80             play      = 1'b0;
81             next_song = 1'b0;
82             reset_play = 1'b0;
83         end
84
85         PLAY: begin
86             play      = 1'b1;
87             next_song = 1'b0;
88             reset_play = 1'b0;
89         end
90
91         NEXT: begin
92             play      = 1'b0;
93             next_song = 1'b1;
94             reset_play = 1'b1;
95         end
96
97         default: begin
98             play      = 1'b0;
99             next_song = 1'b0;
100            reset_play = 1'b0;
101        end
102    endcase
103 end
104
105 // 歌曲序号计数器
106 song_counter u_song_counter (
107     .clk    (clk),
108     .reset  (reset),
109     .en     (next_song),
110     .song   (song)
111 );
112
113 endmodule
```

7.3 乐曲读取模块 song_reader 设计与代码

song_reader 负责在播放过程中从乐曲 ROM 中顺序取出音符信息。该模块输入当前歌曲编号 song[1:0]、播放使能 play 和来自 note_player 的 note_done; 输出当前音符编码 note[5:0]、音符持续时间 duration[5:0]、新音符加载脉冲 new_note 和歌曲结束信号 song_done。

在实现上，**song_reader** 由地址计数器、乐曲 ROM、歌曲结束检测器和控制状态机构成。地址计数器根据当前乐曲编号和当前音符序号生成 7 位 ROM 地址，其中高 2 位对应乐曲编号，低 5 位对应该乐曲内部的音符序号。因此每首歌最多可以包含 $2^5 = 32$ 个音符。**song_rom** 的输出 **dout[11:0]** 被划分为两部分：高 6 位 **dout[11:6]** 为音符编码，低 6 位 **dout[5:0]** 为音符持续时间。

song_reader 的控制状态机主要完成“开始播放—发出新音符—等待当前音符结束—读取下一个音符”的循环。复位后处于 **RESET** 状态；当 **play** 有效时进入 **NEW_NOTE** 状态，并输出一个时钟周期宽度的 **new_note**；随后进入 **WAIT** 状态，等待 **note_done**；当当前音符播放完成后，重新进入 **NEW_NOTE** 状态读取下一音符。若检测到 ROM 地址计数到末尾或读到结束标志，则 **song_end_detector** 输出 **song_done**，反馈给 **mcu**。

Listing 3: song_reader 模块源代码

```
1  module song_reader (
2      input wire      clk,          // 时钟信号
3      input wire      reset,        // 异步复位，高有效
4      input wire      play,         // 播放使能
5      input wire [1:0] song,        // 当前歌曲序号
6      input wire      note_done,    // 当前音符播放完成
7      output wire      song_done,   // 当前歌曲播放完成
8      output wire [5:0] note,       // 当前音符编码
9      output wire [5:0] duration,   // 当前音符持续时间
10     output wire [11:0] dout,       // song_rom 原始输出
11     output reg        new_note     // 新音符加载脉冲
12 );
13
14     wire      co; // 地址计数器进位输出
15     wire [4:0] q; // 当前歌曲内部音符地址
16     wire [6:0] addr; // 拼接后的 song_rom 地址
17
18     localparam RESET      = 2'b00,
19                 NEW_NOTE   = 2'b01,
20                 WAIT       = 2'b10,
21                 NEXT_NOTE  = 2'b11;
22
23     reg [1:0] current_state;
24     reg [1:0] next_state;
25
26     // 状态寄存器
27     always @(posedge clk or posedge reset) begin
28         if (reset)
29             current_state <= RESET;
30         else
31             current_state <= next_state;
32     end
33
34     // 次态逻辑
35     always @(*) begin
36         case (current_state)
37             RESET: begin
```

```
38         if (play)
39             next_state = NEW_NOTE;
40         else
41             next_state = RESET;
42     end
43
44     NEW_NOTE: begin
45         next_state = WAIT;
46     end
47
48     WAIT: begin
49         if (play) begin
50             if (note_done)
51                 next_state = NEW_NOTE;
52             else
53                 next_state = WAIT;
54             end else begin
55                 next_state = RESET;
56             end
57         end
58
59     NEXT_NOTE: begin
60         next_state = NEW_NOTE;
61     end
62
63     default: begin
64         next_state = RESET;
65     end
66 endcase
67 end
68
69 // 输出控制: NEW_NOTE 状态下产生单周期加载脉冲
70 always @(*) begin
71     case (current_state)
72     RESET: begin
73         new_note = 1'b0;
74     end
75
76     NEW_NOTE: begin
77         new_note = 1'b1;
78     end
79
80     WAIT: begin
81         new_note = 1'b0;
82     end
83
84     NEXT_NOTE: begin
85         new_note = 1'b0;
86     end
```

```
87
88     default: begin
89         new_note = 1'b0;
90     end
91 endcase
92 end
93
94 // song_rom 输出格式: [11:6] 为音符, [5:0] 为持续时间
95 assign note      = dout[11:6];
96 assign duration = dout[5:0];
97
98 // 地址计数器: 根据歌曲序号和音符序号生成 ROM 地址
99 addr_counter u_addr_counter (
100     .clk      (clk),
101     .reset     (reset),
102     .note_done (note_done),
103     .song      (song),
104     .q         (q),
105     .addr      (addr),
106     .co        (co)
107 );
108
109 // 歌曲 ROM
110 song_rom u_song_rom (
111     .clk (clk),
112     .addr (addr),
113     .dout (dout)
114 );
115
116 // 歌曲结束检测
117 song_end_detector u_song_end_detector (
118     .clk      (clk),
119     .reset     (reset),
120     .co        (co),
121     .dout      (dout),
122     .song_done (song_done)
123 );
124
125 endmodule
```

7.4 音符播放模块 note_player 设计与代码

note_player 是连接乐曲控制和 DDS 波形产生的关键模块。它接收 song_reader 输出的 note_to_load 和 duration_to_load, 并在 load_new_note 有效时锁存当前音符。其内部主要包含以下部分:

1. 音符播放控制器 note_player_controller: 根据 play_enable、load_new_note 和 timer_done 控制音符装载、计时器清零以及 note_done 输出。
2. 音符定时器 note_timer: 以 beat 为计时基准, 对 duration_to_load 指定的持续时间进行计数, 计时

结束后产生 `timer_done`。

3. 音符寄存器 `note_reg`: 在 `load` 有效时保存当前音符编码, 并作为 `frequency_rom` 的地址。
4. 频率查找表 `frequency_rom`: 根据音符编码输出 20 位频率控制字。由于 DDS 模块使用 22 位相位增量, 代码中将其高位补零, 形成 `k_full={2'b00,dout}`。
5. DDS 模块: 根据补齐后的 22 位频率控制字, 在每个 `sampling_pulse` 到来时输出一个新的 16 位正弦采样值。

当 `play_enable` 为低时, 代码中使用 `R = reset || (!play_enable)` 复位音符寄存器和 DDS, 避免暂停时继续输出旧音符采样。这样可以保证播放状态与音频输出状态一致。

Listing 4: `note_player` 模块源代码

```
1  module note_player (
2      input wire      clk,           // 时钟信号
3      input wire      reset,         // 异步复位, 高有效
4      input wire      play_enable,   // 播放使能
5      input wire [5:0] note_to_load, // 待加载音符编码
6      input wire [5:0] duration_to_load, // 待加载音符持续时间
7      input wire      load_new_note, // 新音符加载请求
8      input wire      sampling_pulse, // 采样脉冲
9      input wire      beat,          // 节拍脉冲
10     output wire     note_done,      // 当前音符播放完成
11     output wire     sample_ready,   // DDS 新采样有效
12     output wire [15:0] sample       // DDS 采样输出
13 );
14
15     wire      timer_clear;
16     wire      load;
17     wire      timer_done;
18     wire [5:0] addr;
19     wire [19:0] dout;
20     wire [21:0] k_full;
21
22     // 播放关闭时复位音符寄存器和 DDS, 避免继续输出旧音符
23     wire R = reset || (!play_enable);
24
25     // 音符播放控制状态机
26     note_player_controller u_controller (
27         .clk      (clk),
28         .reset     (reset),
29         .play_enable (play_enable),
30         .timer_done (timer_done),
31         .load_new_note (load_new_note),
32         .timer_clear (timer_clear),
33         .note_done  (note_done),
34         .load       (load)
35     );
36
37     // 音符持续时间计时器
```

```
38     note_timer u_timer (  
39         .clk          (clk),  
40         .reset         (1'b0),  
41         .timer_clear   (timer_clear),  
42         .beat          (beat),  
43         .duration_to_load (duration_to_load),  
44         .timer_done     (timer_done)  
45     );  
46  
47     // 锁存当前音符编码, 作为 frequency_rom 地址  
48     note_reg u_reg (  
49         .clk          (clk),  
50         .reset        (R),  
51         .load         (load),  
52         .note_to_load (note_to_load),  
53         .q            (addr)  
54     );  
55  
56     // 读取音符对应的频率控制字  
57     frequency_rom u_rom (  
58         .clk (clk),  
59         .addr (addr),  
60         .dout (dout)  
61     );  
62  
63     // frequency_rom 输出为 20 位, DDS 需要 22 位频率控制字  
64     assign k_full = {2'b00, dout};  
65  
66     // DDS 输出对应音符频率的正弦波采样  
67     dds u_dds (  
68         .clk          (clk),  
69         .reset        (R),  
70         .k            (k_full),  
71         .sampling_pulse (sampling_pulse),  
72         .new_sample_ready (sample_ready),  
73         .sample        (sample)  
74     );  
75  
76     endmodule
```

7.5 DDS 模块 dds 设计与代码

dds 模块是 Lab 15 的核心成果, 在音乐播放器中用于将离散音符编码转化为相应频率的正弦波采样。上传代码中的 DDS 模块采用 22 位相位累加器, 输入频率控制字为 $k[21:0]$ 。每当 `sampling_pulse` 有效时, 相位寄存器更新为

$$\text{raw_addr} \leftarrow \text{raw_addr} + k.$$

随后, `addr_process` 根据相位地址所在象限将完整周期地址映射到四分之一周期的 `sine_rom` 地址; `sine_rom` 输出第 0 象限的 16 位正弦样品; `data_process` 再根据相位最高位判断当前位于正半周还是负半周, 对数据进行符号处理。最后, 输出寄存器在采样脉冲控制下锁存 `sample`, 同时将 `sampling_pulse` 延迟一拍得到 `new_sample_ready`。

这种设计利用正弦波四分之一周期对称性减少 ROM 存储量, 并通过 22 位相位累加器提高频率控制精度。由于音乐播放器中的音符频率由 `frequency_rom` 给出, 因此同一个 DDS 模块可以复用于所有音符的正弦波生成。

Listing 5: dds 模块源代码

```
1  module dds (
2      input wire      clk,           // 时钟信号
3      input wire      reset,         // 异步复位, 高有效
4      input wire [21:0] k,           // 频率控制字
5      input wire      sampling_pulse, // 采样使能脉冲
6      output wire     new_sample_ready, // 新采样数据有效标志
7      output wire [15:0] sample      // 输出正弦波采样值
8  );
9
10 // 内部连线
11 wire [21:0] raw_addr; // 相位累加器当前值
12 wire [21:0] adder_out; // 相位累加器加法结果
13 wire [9:0] rom_addr; // 正弦 ROM 地址
14 wire [15:0] raw_data; // ROM 输出的原始数据
15 wire [15:0] data; // 符号处理后的数据
16 wire      area_bit; // 延迟一拍后的相位最高位
17
18 // 相位累加器: raw_addr <= raw_addr + k
19 adder_22 u_adder (
20     .a (k),
21     .b (raw_addr),
22     .sum (adder_out)
23 );
24
25 dff_en_rst #(
26     .WIDTH (22)
27 ) u_phase_reg (
28     .clk (clk),
29     .reset (reset),
30     .en (sampling_pulse),
31     .d (adder_out),
32     .q (raw_addr)
33 );
34
35 // 地址处理: 根据象限完成地址截取/镜像
36 addr_process u_addr_process (
37     .raw_addr (raw_addr),
38     .rom_addr (rom_addr)
```

```

39     );
40
41     // 正弦 ROM: 输出 1/4 周期波形数据
42     sine_rom u_sine_rom (
43         .clk   (clk),
44         .addr  (rom_addr),
45         .dout  (raw_data)
46     );
47
48     // 相位最高位打一拍, 与 ROM 输出数据时序对齐
49     dff_en_rst #(
50         .WIDTH (1)
51     ) u_area_bit_reg (
52         .clk   (clk),
53         .reset (reset),
54         .en    (1'b1),
55         .d     (raw_addr[21]),
56         .q     (area_bit)
57     );
58
59     // 根据相位区域生成正/负半周期数据
60     data_process u_data_process (
61         .raw_data (raw_data),
62         .area_bit (area_bit),
63         .data     (data)
64     );
65
66     // 输出采样寄存器
67     dff_en_rst #(
68         .WIDTH (16)
69     ) u_sample_reg (
70         .clk   (clk),
71         .reset (reset),
72         .en    (sampling_pulse),
73         .d     (data),
74         .q     (sample)
75     );
76
77     // 采样有效标志打一拍输出
78     dff_en_rst #(
79         .WIDTH (1)
80     ) u_ready_reg (
81         .clk   (clk),
82         .reset (reset),
83         .en    (1'b1),
84         .d     (sampling_pulse),
85         .q     (new_sample_ready)
86     );
87

```

88 endmodule

7.6 采样同步与节拍产生模块设计与代码

音频接口模块和系统控制逻辑可能处在不同的时序关系下, 因此 `music_player` 中专门使用 `sync_pulse` 对 `NewFrame` 信号进行同步和边沿检测。`sync_pulse` 先用两级寄存器将 `NewFrame` 同步到 `clk` 时钟域, 再在检测到上升沿时输出单周期 `ready` 脉冲。该 `ready` 脉冲作为 `note_player` 和 DDS 的 `sampling_pulse`, 表示音频接口需要一个新的采样点。

节拍信号 `beat` 由 `divider` 模块产生, 用于控制音符持续时间。`divider` 是参数化分频器, 当计数值达到 `n-1` 时输出一个时钟周期的 `beat` 脉冲。代码中设置

$$n = \begin{cases} 64, & \text{仿真模式}(\text{sim} = 1), \\ 1000, & \text{硬件模式}(\text{sim} = 0), \end{cases}$$

其中仿真模式使用较小分频系数, 可以明显缩短仿真时间; 硬件模式下若系统时钟为 100 MHz, 则 `beat` 频率约为 50 Hz, 用于作为音符定时基准。

Listing 6: `sync_pulse` 模块源代码

```

1 module sync_pulse (
2     input wire clk,    // 时钟信号
3     input wire reset,  // 异步复位, 高有效
4     input wire in,     // 异步输入信号
5     output reg ready   // 同步后的单周期上升沿脉冲
6 );
7
8     reg [1:0] sync_reg; // 两级同步寄存器
9     reg      in_prev;   // 上一拍同步信号, 用于边沿检测
10
11     always @(posedge clk or posedge reset) begin
12         if (reset) begin
13             sync_reg <= 2'b00;
14             in_prev  <= 1'b0;
15             ready    <= 1'b0;
16         end else begin
17             sync_reg <= {sync_reg[0], in};
18             in_prev  <= sync_reg[1];
19             ready    <= sync_reg[1] & ~in_prev;
20         end
21     end
22
23 endmodule

```

Listing 7: `divider` 模块源代码

```

1 module divider #(
2     parameter n          = 1000, // 分频计数值

```



```

3     parameter counter_bits = 10    // 计数器位宽
4 ) (
5     input wire clk,    // 时钟信号
6     input wire reset,  // 异步复位, 高有效
7     output reg beat    // 分频输出脉冲
8 );
9
10    reg [counter_bits-1:0] cnt;
11
12    always @(posedge clk or posedge reset) begin
13        if (reset) begin
14            cnt <= {counter_bits{1'b0}};
15            beat <= 1'b0;
16        end else begin
17            if (cnt == n - 1) begin
18                cnt <= {counter_bits{1'b0}};
19                beat <= 1'b1;
20            end else begin
21                cnt <= cnt + 1'b1;
22                beat <= 1'b0;
23            end
24        end
25    end
26
27 endmodule

```

7.7 顶层模块 music_player 设计与代码

music_player 是播放器核心顶层模块, 内部完成采样同步、节拍产生、主控制、乐曲读取与音符播放等模块的连接。该模块接收已经经过按键处理后的 **play_pause** 与 **next** 单周期脉冲, 同时接收音频接口给出的 **NewFrame** 信号。**sync_pulse** 将 **NewFrame** 转换为系统时钟域的 **ready** 脉冲, **divider** 产生音符节拍 **beat**; **mcu** 根据按键和歌曲结束信号控制当前播放状态与歌曲序号; **song_reader** 读取当前歌曲的 **note** 和 **duration**; **note_player** 根据音符信息调用 DDS 产生 **sample** 输出。

Listing 8: music_player 顶层模块源代码

```

1  `timescale 1ns / 1ps
2
3  module music_player #(
4      parameter sim = 0 // 仿真模式: 1 时使用较快节拍
5  ) (
6      input wire      clk,          // 时钟信号
7      input wire      reset,        // 异步复位, 高有效
8      input wire      play_pause,   // 播放/暂停切换脉冲
9      input wire      next,         // 下一首切换脉冲
10     input wire      NewFrame,     // 音频接口提供的异步采样帧信号
11     output wire [15:0] sample,    // 正弦波采样输出
12     output wire      play,        // 播放状态

```

```

13     output wire [1:0] song           // 当前歌曲序号
14 );
15
16     wire      ready;                // 同步后的采样脉冲
17     wire      beat;                 // 节拍基准脉冲
18     wire      reset_play;          // 播放链路复位信号
19     wire      song_done;           // 歌曲播放完成标志
20     wire      note_done;           // 当前音符播放完成标志
21     wire      new_note;             // 新音符加载脉冲
22     wire [5:0] note;                // 当前音符编码
23     wire [5:0] duration;           // 当前音符持续时间
24
25     // 将音频接口的异步 NewFrame 转换为系统时钟域内的单周期脉冲
26     sync_pulse u_sync_pulse (
27         .clk      (clk),
28         .reset     (reset),
29         .in        (NewFrame),
30         .ready     (ready)
31     );
32
33     // 节拍基准分频器
34     divider #(
35         .n          (sim ? 64 : 1000),
36         .counter_bits (sim ? 6 : 10)
37     ) u_divider (
38         .clk      (clk),
39         .reset     (reset),
40         .beat     (beat)
41     );
42
43     // 主控制器
44     mcu u_mcu (
45         .clk      (clk),
46         .reset     (reset),
47         .play_pause (play_pause),
48         .next      (next),
49         .song_done  (song_done),
50         .play      (play),
51         .song      (song),
52         .reset_play (reset_play)
53     );
54
55     // 读取当前歌曲的音符和时长
56     song_reader u_song_reader (
57         .clk      (clk),
58         .reset     (reset || reset_play),
59         .play      (play),
60         .song      (song),
61         .note_done (note_done),

```

```
62     .song_done (song_done),
63     .note      (note),
64     .duration   (duration),
65     .dout       (),          // 调试输出未使用
66     .new_note   (new_note)
67 );
68
69 // 根据当前音符生成对应频率的采样数据
70 note_player u_note_player (
71     .clk          (clk),
72     .reset         (reset || reset_play),
73     .play_enable   (play),
74     .note_to_load  (note),
75     .duration_to_load (duration),
76     .load_new_note (new_note),
77     .sampling_pulse (ready),
78     .beat          (beat),
79     .note_done     (note_done),
80     .sample_ready  (),      // 调试输出未使用
81     .sample        (sample)
82 );
83
84 endmodule
```

7.8 复位与模块间握手机制

系统复位分为全局复位和播放链路复位两类。全局复位 `reset` 来自外部按键或顶层复位信号，作用于整个播放器；播放链路复位 `reset_play` 由 `mcu` 产生，主要在系统复位、切换下一首和歌曲播放结束时清空 `song_reader` 与 `note_player` 的内部状态。在 `music_player` 中，`song_reader` 和 `note_player` 的复位端均接为 `reset || reset_play`，从而保证切歌或重新播放时音符地址、音符寄存器和 DDS 相位累加器都从初始状态开始。

模块之间的握手关系如下：`mcu` 输出 `play` 允许 `song_reader` 开始读取音符；`song_reader` 输出 `new_note` 通知 `note_player` 装载新的 `note` 和 `duration`；`note_player` 在当前音符定时结束后输出 `note_done`，请求 `song_reader` 读取下一音符；当 `song_reader` 判断当前歌曲结束时输出 `song_done`，通知 `mcu` 回到复位/暂停流程。该握手机制使乐曲读取速度由音符实际播放时间决定，而不是简单地按固定时钟连续读取 ROM。

八、仿真调试与数据记录

8.1 Lab 11 按键处理仿真

测试平台 `button_press_tb` 中设置 `delay=10`，时钟信号由

```
always #(delay/2) clk=~clk;
```

产生，因此仿真时钟周期为 10 ns。仿真开始时 `reset` 置为高电平，模块输出被复位为低电平；经过 3 个

时钟周期后释放复位。随后，测试平台通过多组 `repeat(25)` 语句使 `ButtonIn` 在 0 和 1 之间快速翻转，用于模拟机械按键按下和释放过程中的不稳定变化；在两组快速翻转之间加入较长延时，用于模拟按键稳定保持高电平或低电平的过程。为了缩短消抖等待时间，例化 `button_unit` 时令参数 `sim=1`，从而便于在较短仿真时间内观察输出响应。

图7给出了按键处理模块的 ModelSim 仿真结果。波形中自上而下依次为 `clk`、`ButtonIn`、`reset` 和 `ButtonOut`。

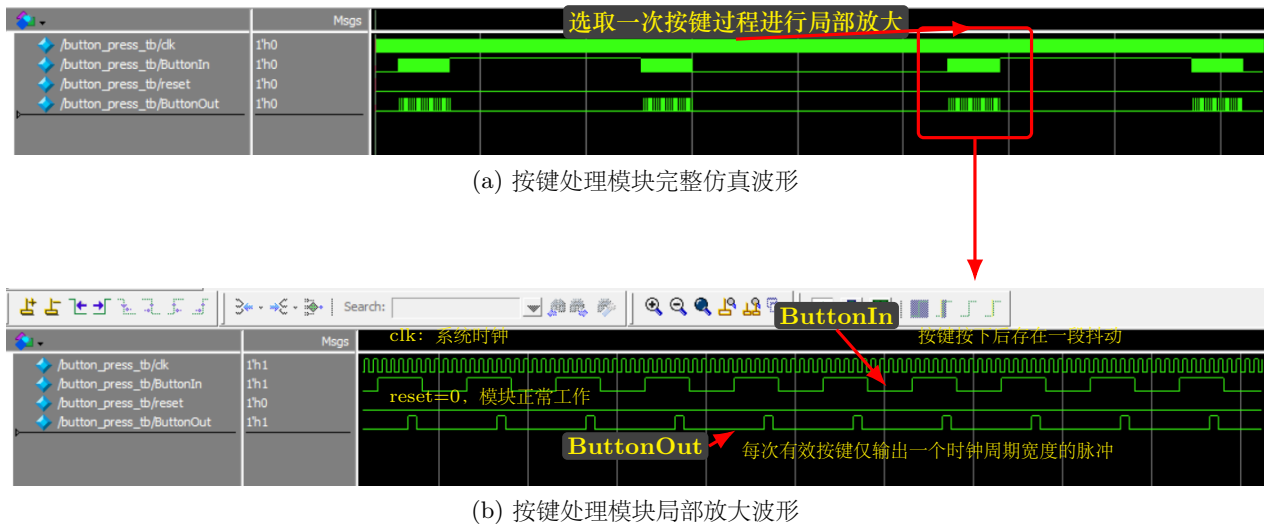


图 7: 按键处理模块 ModelSim 仿真波形及局部放大分析

由完整波形可以看出，在仿真初始阶段 `reset=1` 时，`ButtonOut` 保持为低电平，说明复位功能正常。复位释放后，`ButtonIn` 按照 testbench 的设置出现若干组快速翻转，并在每组翻转后保持稳定状态。`ButtonOut` 不直接保持为高电平，而是在检测到一次有效按键动作后输出一个窄脉冲，因此该输出更适合作为后级状态机的控制信号。

由局部放大波形可以进一步看出，`clk` 连续翻转作为采样时钟；当 `ButtonIn` 出现一次有效上升沿并经过模块处理后，`ButtonOut` 只在一个时钟周期内为高电平，随后立即回到低电平。也就是说，模块并没有把按键保持时间原样传递给后级电路，而是将一次按键动作转换为一个 `clk` 周期宽度的脉冲。这样可以避免后级 `mcu` 在按键保持期间重复响应同一次按键操作。

结合 `button_unit` 的结构可知，`ButtonIn` 首先经过同步触发器进入系统时钟域，降低异步输入直接作用于时序电路时产生亚稳态的概率；随后经过消抖滤波器，滤除按键按下或释放过程中的不稳定变化；最后通过寄存上一拍消抖信号并与当前消抖信号进行组合逻辑比较，实现上升沿检测。仿真结果表明，`button_unit` 能够在有效按键输入到来时产生单周期 `ButtonOut` 脉冲，满足音乐播放器中 `play/pause_button` 和 `next_button` 对可靠按键控制信号的要求。

8.2 DDS 模块仿真分析

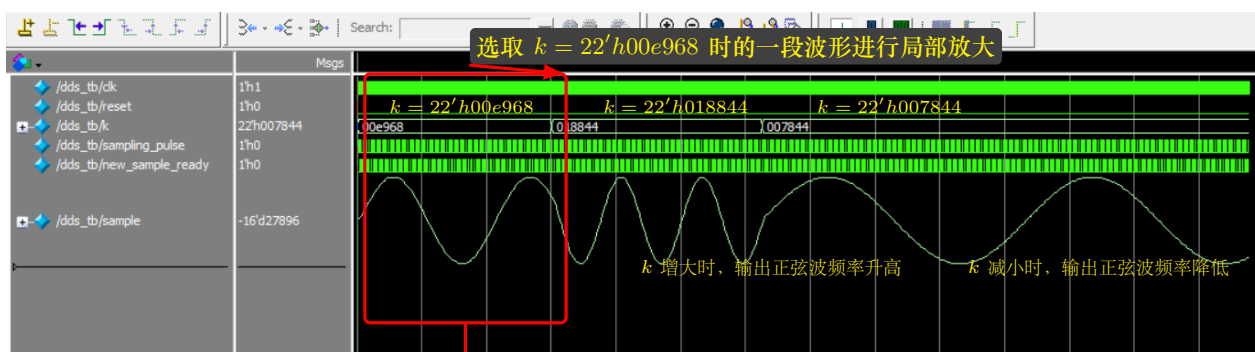
DDS 模块的作用是在采样脉冲 `sampling_pulse` 的控制下，根据输入的相位增量 `k` 产生对应频率的正弦波采样序列，并在新的采样点产生后给出 `new_sample_ready` 信号。仿真过程中首先将 `reset` 置为高电平，对 DDS 模块进行复位；随后释放复位，并依次设置三组不同的相位增量：

$$k_1 = \{12'd58, 10'd360\} = 22'h00e968,$$

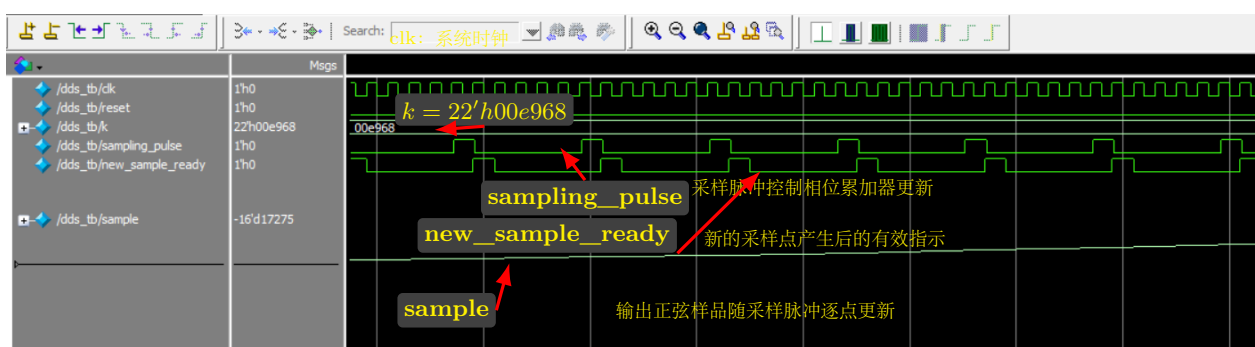
$$k_2 = \{12'd98, 10'd68\} = 22'h018844,$$

$$k_3 = \{12'd30, 10'd68\} = 22'h007844.$$

在 DDS 中，输出正弦波频率与相位增量 k 成正比，因此当 k 增大时，输出波形频率升高；当 k 减小时，输出波形频率降低。测试中通过改变 k 的值，可以观察 DDS 输出波形频率随相位增量变化的情况。



(a) DDS 模块完整仿真波形



(b) DDS 模块局部放大波形

图 8: DDS 模块 ModelSim 仿真波形及局部放大分析

图8给出了 DDS 模块的 ModelSim 仿真结果。完整波形中，输入相位增量 k 依次取 $22'h00e968$ 、 $22'h018844$ 和 $22'h007844$ 。可以观察到，当 k 从 $22'h00e968$ 增大到 $22'h018844$ 时，输出 `sample` 的波形周期变短，频率明显升高；当 k 减小到 $22'h007844$ 后，输出波形周期变长，频率降低。这与 DDS 输出频率和相位增量成正比的理论关系一致。

局部放大图中固定 $k=22'h00e968$ ，可以更清楚地观察 `sampling_pulse`、`new_sample_ready` 与 `sample` 之间的时序关系。每当 `sampling_pulse` 有效时，DDS 模块进行一次相位累加，并从正弦查找表中读取相应的正弦样品；随后 `new_sample_ready` 给出有效指示，表示新的采样数据已经产生。与此同时，`sample` 输出呈阶梯状逐点变化，说明 DDS 模块不是连续模拟输出，而是在采样脉冲控制下逐点产生离散的正弦波采样序列。

综上，仿真结果表明：DDS 模块能够在采样脉冲控制下正确输出正弦波样品，并且输出正弦波频率能够随相位增量 k 的变化而改变。该模块满足音乐播放器实验中根据不同音符频率产生对应正弦波采样数据的设计要求。

8.3 MCU 主控制器模块仿真分析

MCU 主控制器模块是音乐播放器的控制核心，其主要功能是根据外部按键信号和乐曲播放状态，对播放器的播放、暂停、切歌以及播放链路复位进行控制。模块输入信号包括系统时钟 `clk`、复位信号 `reset`、播放/暂停按键脉冲 `play_pause`、下一首按键脉冲 `next` 以及当前乐曲播放完成信号 `song_done`；输出信号包括播放使能信号 `play`、当前乐曲序号 `song` 以及播放复位信号 `reset_play`。

本模块的仿真测试主要通过依次施加 `reset`、`play_pause`、`next` 和 `song_done` 信号，观察 `play`、`song` 和 `reset_play` 的变化情况。测试过程首先对模块进行复位，然后多次输入播放/暂停脉冲，验证 `play` 信号能否在播放与暂停状态之间正确切换；之后输入 `next` 脉冲，验证歌曲序号是否能够递增，同时观察切歌时是否产生 `reset_play` 脉冲；最后输入 `song_done` 信号，验证当前乐曲播放完成后控制器能否重新复位播放链路。

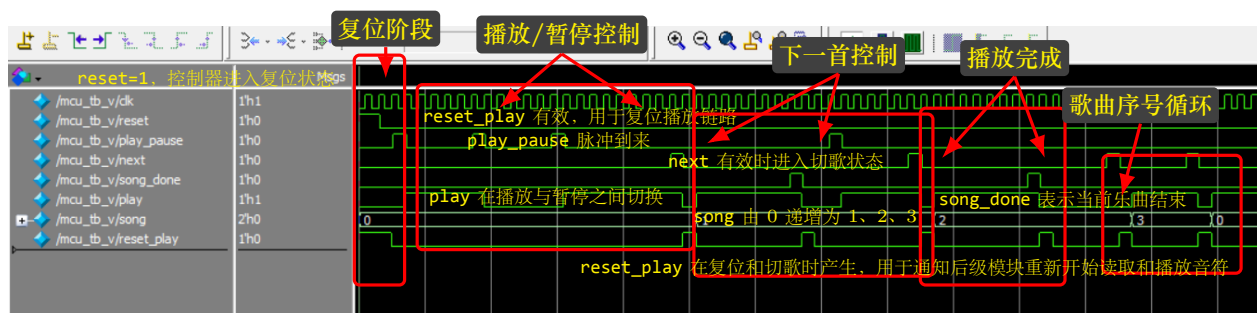


图 9: MCU 主控制器模块 ModelSim 仿真波形

图9给出了 MCU 主控制器模块的 ModelSim 仿真波形。仿真开始时，`reset` 被置为高电平，模块进入复位状态，此时 `play` 为低电平，表示播放器处于未播放状态；同时 `reset_play` 有效，用于通知后级的乐曲读取模块和音符播放模块清除当前播放状态。

当 `reset` 释放后，输入 `play_pause` 脉冲，控制器响应播放/暂停按键操作。由波形可见，每当 `play_pause` 出现有效脉冲时，输出 `play` 在高、低电平之间切换：`play=1` 时表示播放器处于播放状态，`play=0` 时表示播放器处于暂停状态。这说明主控制器能够正确完成播放与暂停之间的状态转换。

当输入 `next` 脉冲时，控制器进入切歌状态。此时 `song` 按照二位二进制计数方式递增，波形中可以看到歌曲序号依次由 0 变为 1、2、3，随后再次回到 0，说明歌曲序号计数器能够实现四首歌曲之间的循环切换。同时，在切歌过程中 `reset_play` 会产生有效脉冲，用于复位后级播放链路，使 `song_reader` 和 `note_player` 能够从新歌曲的起始位置重新开始工作。

当 `song_done` 产生有效脉冲时，表示当前乐曲已经播放结束。由仿真波形可见，控制器能够响应该信号并产生相应的复位控制，使系统重新回到可继续播放或切换歌曲的状态。综上所述，MCU 模块能够正确响应播放/暂停、下一首以及乐曲播放完成信号，输出的 `play`、`song` 和 `reset_play` 均符合音乐播放器主控制器的设计要求。

8.4 song_reader 乐曲读取模块仿真分析

`song_reader` 模块用于根据主控制器给出的播放控制信号和歌曲序号，从乐曲存储器中依次读出当前歌曲的音符信息，并将音符标号 `note` 和音长 `duration` 送往后级的 `note_player` 模块。当 `note_player` 完成一个音符的播放后，会通过 `note_done` 向 `song_reader` 请求下一个音符；`song_reader` 在读出新的音符信息后

产生 **new_note** 脉冲，通知后级模块加载新的音符。当读到乐曲结束标志时，模块输出 **song_done**，通知主控制器当前乐曲播放结束。

本模块的仿真测试主要通过设置 **reset**、**play**、**song** 和 **note_done** 信号，观察 **note**、**duration**、**new_note** 和 **song_done** 的变化情况。仿真开始时先对模块复位，随后使能 **play**，并固定 **song=2'd0**，表示读取第 0 首乐曲；之后周期性产生 **note_done** 脉冲，用于模拟后级音符播放模块不断请求新音符的过程。

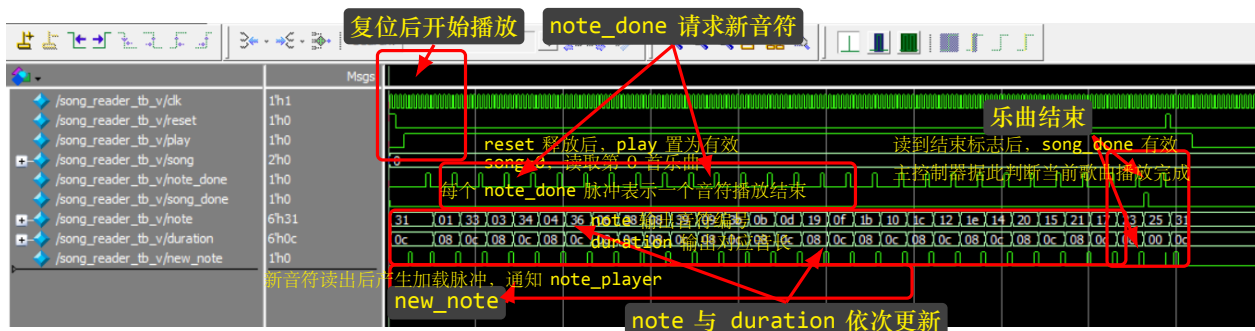


图 10: song_reader 乐曲读取模块 ModelSim 仿真波形

图10给出了 **song_reader** 模块的 ModelSim 仿真结果。仿真开始时，**reset** 短暂置为高电平，模块被复位；复位释放后，**play** 被置为高电平，表示系统进入播放状态。此时 **song** 保持为 0，说明当前仿真读取的是第 0 首乐曲。

在播放过程中，**note_done** 周期性产生脉冲，用来模拟后级 **note_player** 已经完成当前音符播放并请求下一个音符。由波形可以看出，每当 **note_done** 有效后，**song_reader** 都会继续从乐曲 ROM 中读出下一组音符信息，**note** 和 **duration** 随之更新。例如，波形中 **note** 依次输出 31、01、33、03、34 等音符编号，**duration** 则同步给出对应的音长信息，如 0c、08 等。

同时，在每次新的音符信息读出后，模块会产生 **new_note** 脉冲。该信号作为后级 **note_player** 的加载控制信号，用于通知其读取当前的 **note** 和 **duration**，并开始播放新的音符。由仿真波形可见，**new_note** 与音符读取过程保持同步，能够正确反映新音符已经准备好的时刻。

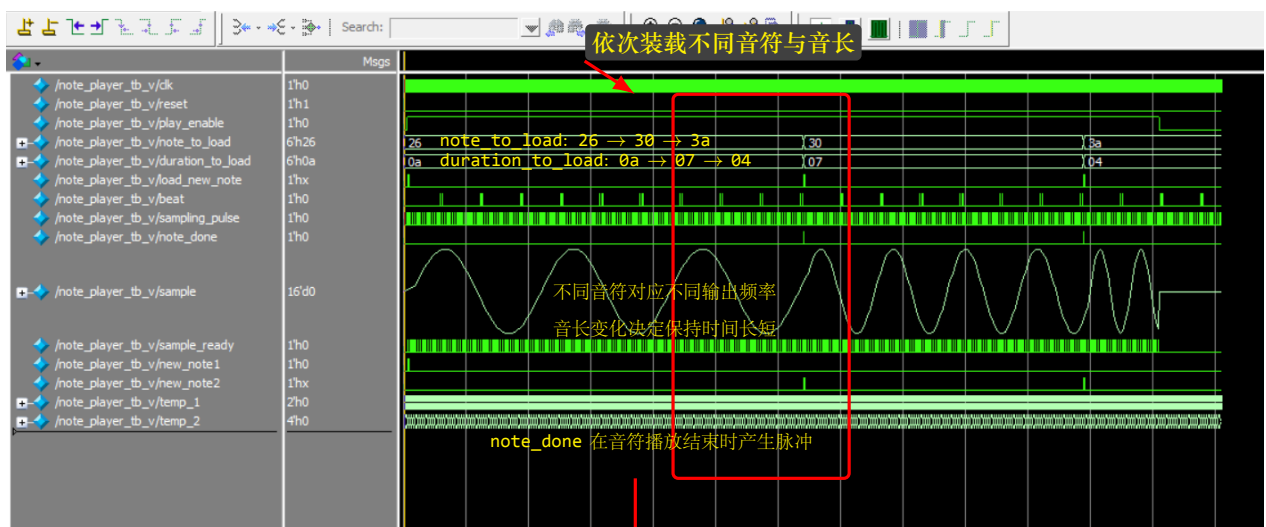
在波形后段，当乐曲读取到结束标志时，**song_done** 被置为有效，表示当前歌曲已经播放完毕。随后主控制器即可根据该信号执行后续控制，例如停止播放、重新复位播放链路或切换到下一首歌曲。综上，**song_reader** 模块能够在 **play** 有效时根据 **note_done** 请求依次读取音符和音长，并在乐曲结束时正确产生 **song_done** 信号，满足音乐播放器中乐曲读取模块的设计要求。

8.5 note_player 音符播放模块仿真分析

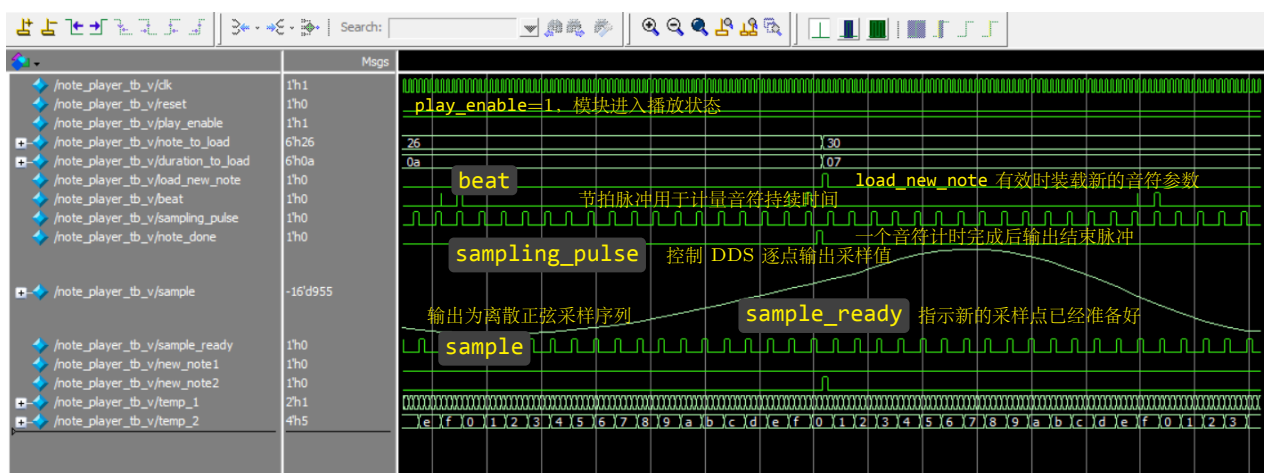
note_player 模块是音乐播放器中的核心模块之一，其主要功能是根据输入的音符编号 **note_to_load** 和音长 **duration_to_load**，查找对应的频率控制字，并驱动 DDS 模块产生该音符对应的正弦波采样序列。同时，该模块还需要依据节拍信号 **beat** 控制音符持续时间，并在一个音符播放结束后输出 **note_done** 脉冲，通知前级模块读取下一个音符。

本模块的仿真测试主要通过设置 **reset**、**play_enable**、**note_to_load**、**duration_to_load** 和 **load_new_note** 等信号，观察 **sample**、**sample_ready** 以及 **note_done** 的变化情况。仿真开始时先对模块进行复位，随后使能 **play_enable**，并依次装载多组不同的音符与音长参数，例如波形中可以看到 **note_to_load** 先后取 26、30、3a，对应的 **duration_to_load** 分别取 0a、07、04，从而验证模块在不同音高、不同持续时间条件下的工作

情况。



(a) note_player 模块完整仿真波形



(b) note_player 模块局部放大波形

图 11: note_player 音符播放模块 ModelSim 仿真波形及局部放大分析

图 11 给出了 note_player 模块的 ModelSim 仿真结果。由完整波形可以看出，在 play_enable 有效后，模块能够根据输入的 note_to_load 和 duration_to_load 装载新的音符信息。随着输入音符由 26 变为 30，再变为 3a，输出信号 sample 的波形频率也随之发生变化，说明模块能够根据不同音符查找相应的频率控制字，并生成对应频率的正弦波采样序列。同时，音长由 0a、07 到 04 的变化，也反映在不同音符保持时间的长短变化上。

局部放大图进一步展示了该模块的关键时序关系。首先，beat 信号作为节拍基准脉冲，用于对音符持续时间进行计数；当节拍计数达到预设音长后，模块会判定当前音符播放结束，并输出 note_done 脉冲。其次，sampling_pulse 用于驱动 DDS 模块按固定采样速率更新输出，因此 sample 呈现出离散采样点构成的正弦波形，而不是连续模拟波形。与此同时，sample_ready 与采样输出过程同步变化，用于指示新的采样点已经有效，可供后级音频接口模块读取。

此外，从波形中还可以观察到，当 load_new_note 有效时，模块会装载新的 note_to_load 和 duration_to_load，

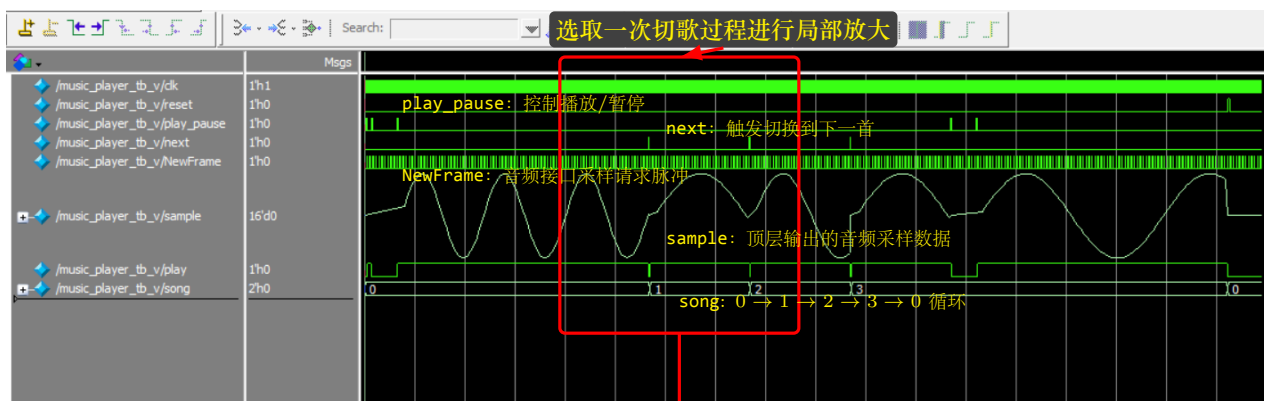
并在后续播放过程中切换到新的输出频率。这说明 **note_player** 模块不仅能够持续输出当前音符的采样序列，还能够收到新的音符装载请求后，及时完成音符切换。

综上所述，**note_player** 模块能够正确完成音符加载、音长计时、正弦波采样输出以及音符结束指示等功能；其输出波形频率能够随输入音符变化而改变，且能够在音符持续时间结束时产生 **note_done** 信号，满足音乐播放器实验对音符播放模块的设计要求。

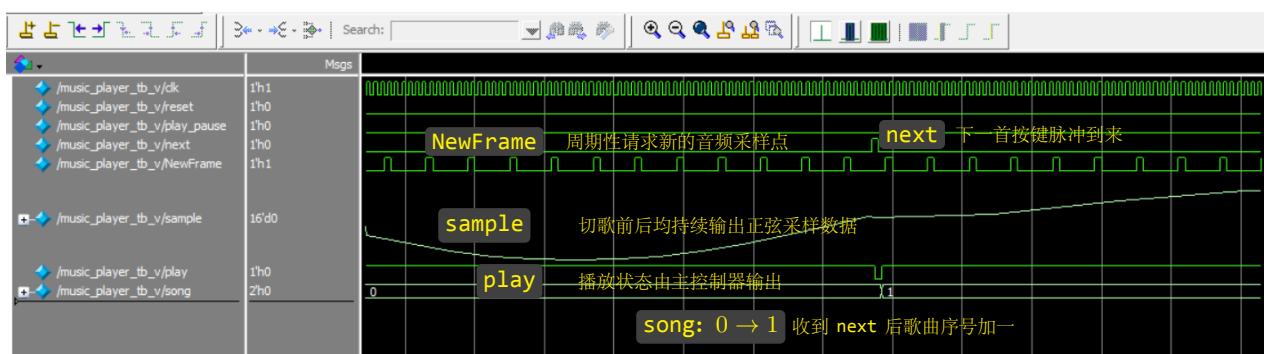
8.6 music_player 顶层模块仿真分析

music_player 是本实验音乐播放器系统的顶层模块，内部连接了主控制器 **mcu**、乐曲读取模块 **song_reader**、音符播放模块 **note_player**、DDS 正弦波发生模块以及采样同步相关电路。该模块对外接收系统时钟 **clk**、复位信号 **reset**、播放/暂停控制信号 **play_pause**、下一首控制信号 **next** 和音频接口采样帧信号 **NewFrame**，并输出音频采样数据 **sample**、播放状态 **play** 和当前乐曲序号 **song**。

本模块的仿真测试主要用于验证整个音乐播放器系统的综合工作过程。测试中首先对系统进行复位，然后通过 **play_pause** 脉冲启动播放，再通过 **next** 脉冲依次切换歌曲，同时观察 **sample** 是否能够持续输出正弦波采样序列，**play** 是否能够正确反映播放/暂停状态，**song** 是否能够按照 $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 0$ 的顺序循环变化。仿真中还周期性产生 **NewFrame** 信号，用于模拟音频编解码接口向播放器请求新采样点的过程。



(a) **music_player** 顶层模块完整仿真波形



(b) **music_player** 顶层模块局部放大波形

图 12: **music_player** 顶层模块 ModelSim 仿真波形及局部放大分析

图 12 给出了 **music_player** 顶层模块的 ModelSim 仿真结果。由完整波形可以看出，在系统复位释放后，

输入 **play_pause** 脉冲能够控制播放器进入播放状态，输出 **play** 随之有效，表示系统开始播放音乐。在播放过程中，**sample** 输出连续变化的正弦波采样数据，说明顶层模块能够正确驱动内部音符播放模块和 DDS 模块产生音频采样序列。

仿真中多次施加 **next** 脉冲，用于模拟用户按下“下一首”按键。由波形可见，每当 **next** 有效后，输出 **song** 会随之改变，歌曲序号依次按照 $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 0$ 的顺序循环变化，说明顶层模块中的主控制器能够正确完成歌曲切换控制。同时，在歌曲切换前后，**sample** 仍能继续输出正弦波采样数据，只是由于切换到了新的乐曲或新的音符，输出波形的频率和形态会发生相应变化。

局部放大图进一步展示了一次切歌过程中的关键信号关系。在 **NewFrame** 周期性产生的过程中，系统不断向后级音频接口提供新的 **sample** 采样值；当 **next** 脉冲到来后，**song** 由 0 变为 1，说明系统已经从第 0 首歌曲切换到第 1 首歌曲。切歌过程中，**play** 保持有效，表明系统在播放状态下可以直接切换歌曲，不需要先暂停再切歌。

此外，仿真后段还可以看到，当 **play_pause** 再次产生脉冲时，**play** 会发生变化，说明顶层模块能够响应播放/暂停控制信号；而当歌曲序号到达 3 后再次切歌时，**song** 回到 0，说明歌曲序号计数具有循环特性。综上所述，**music_player** 顶层模块能够正确连接各子模块，实现播放/暂停控制、歌曲循环切换、采样脉冲响应和音频采样输出等功能，满足音乐播放器系统的整体设计要求。

九、下载验证与实验结果分析

9.1 开发板验证步骤

1. 在 Vivado 中完成综合、实现和比特流生成。
2. 检查 **clk**、**reset**、按键、LED 和音频接口的引脚约束是否正确。
3. 下载比特流到 FPGA 开发板。
4. 按下 **play/pause_button**，观察 LED0 是否随播放/暂停状态变化。
5. 按下 **next_button**，观察 LED2、LED3 是否切换乐曲序号，并检查播放曲目是否改变。
6. 使用耳机或扬声器确认音乐播放是否连续、音高是否基本正确、暂停和切歌是否响应正常。

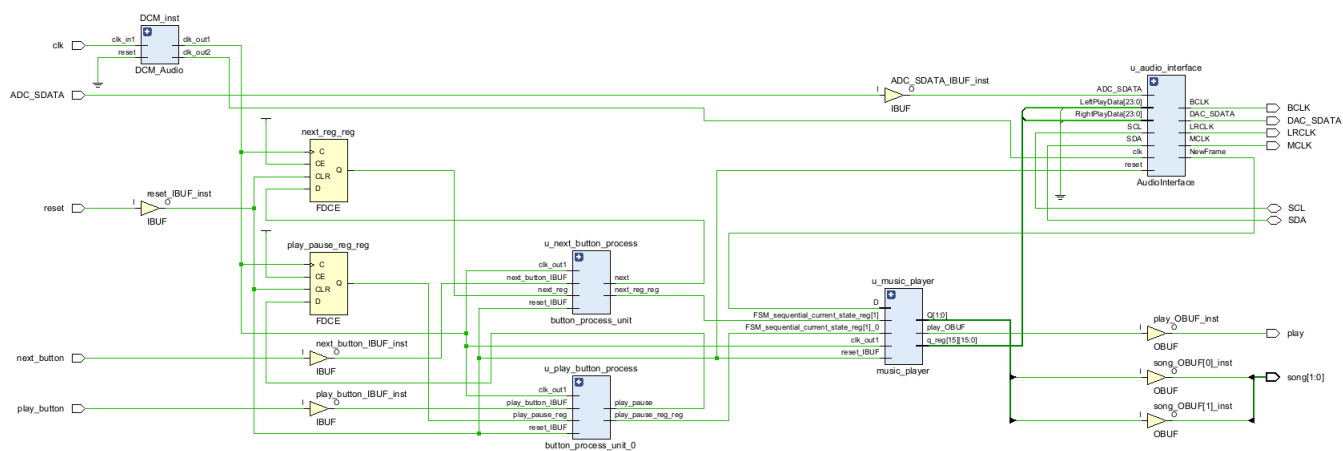


图 13: Vivado 原理图

9.2 实验结果记录

测试项目	预期结果	实际结果
播放/暂停按钮	按一次切换播放状态，LED0 同步变化	正常变化
下一首按钮	按一次切换到下一首，LED2/LED3 显示改变	正常显示乐曲编号（二进制）
复位按钮	系统回到初始状态	正确复位
四首乐曲播放	可顺序播放四首乐曲	可以实现四首音乐循环播放
音频输出	扬声器/耳机能听到对应音符旋律	可以听到正常的乐曲

表 4: 开发板下载验证结果记录表

9.3 结果分析

从开发板验证结果来看，本次音乐播放器系统能够完成实验要求中的基本功能。按下 `play/pause_button` 后，系统可以在播放和暂停状态之间切换，LED0 能够同步指示当前播放状态，说明主控制器对播放/暂停按钮的响应正确。按下 `next_button` 后，LED2 和 LED3 显示的歌曲序号能够发生变化，并且实际播放曲目随之切换，说明歌曲选择计数和乐曲读取模块工作正常。

音频输出方面，耳机或扬声器中可以听到较为连续的乐曲旋律，说明 `song_reader` 能够正确读出音符和音长，`note_player` 能够根据音符信息调用 DDS 模块产生相应频率的正弦采样数据，音频接口也能够将采样数据输出到编解码芯片。四首乐曲可以循环播放，表明歌曲序号循环、乐曲结束判断以及切歌复位逻辑均能正常工作。

实验过程中没有出现明显的按钮误触发或连续触发问题，说明按钮同步与防颤处理有效。复位按钮按下后，系统能够回到初始状态，说明各主要模块的复位逻辑正确。总体来看，本设计在仿真和板级验证中均能实现播放/暂停、下一首切换、四首乐曲循环播放和音频输出等功能，满足本次音乐播放器实验的任务要求。

十、实验心得

通过本次音乐播放器实验，我对数字系统“自顶而下”的设计方法有了更直观的认识。整个系统由按钮处理、主控制器、乐曲读取、音符播放、DDS 正弦波发生和音频接口等多个模块组成，只有各模块之间的控制信号和时序关系正确，系统才能正常工作。因此，在设计过程中不仅要关注单个模块的功能，还要重视模块之间的连接和握手关系。

本实验也加深了我对 DDS 技术的理解。通过改变相位增量，可以控制输出正弦波的频率，从而产生不同音高的音符。相比直接存储大量波形数据，DDS 方法结构清晰、控制方便，也体现了数字电路在信号生成方面的灵活性。

在实验过程中，我也遇到了不少的问题，在解决问题的同时也锻炼了我校验代码与使用外部工具的能力：

1. DDS 模块的波形问题：在 ModelSim 进行 DDS 模块的仿真时，波形出现了突上突下的跳跃，与预期的正弦波不符

问题与解决方法：显示的数字格式不为十进制，导致在负数时会被折射为正数，将显示格式改为十进制即可

2. 乐曲播放过快的问题：在第一次检验的时候，出现了乐曲播放过快的问题

问题与解决方法：在阅读材料的时候没有注意到分频器要求分频比 1000，因此在调用顶层模块时参数填写错误，更正即可

3. 乐曲切换时无法从头播放：在切换到下一首歌的时候出现了无法从下一首乐曲的开头播放，而是从上一首歌的结束地址开始播放

问题与解决方法：问题有两个，第一个是在顶层模块时，没有把 `reset_play` 作为复位信号之一；第二个是在 `mcu` 模块中的 `NEXT` 状态里，`reset_play` 错误的置 0。在解决的过程中，我使用了 AI 工具。由于文件数量巨大，一个个检查起来会耗费大量时间，因此使用了 AI 检查了逻辑合理性，最终发现了 `mcu` 模块的编写错误。

总体来说，本次实验综合性较强，涉及状态机设计、ROM 查表、计数分频、跨模块控制以及板级验证等内容。通过完成该实验，我对 FPGA 数字系统设计流程有了更完整的理解，也提高了 Verilog 编程、模块化设计和仿真调试能力。

十一、思考题

1. 为什么 `play/pause` 和 `next` 按键需要防颤动及同步化处理，而 `reset` 按键通常可不作同样处理？

答：`play/pause` 和 `next` 按键属于功能控制按键，每按下一次都应该只产生一次有效操作。如果不进行防颤动处理，机械按键在按下或释放瞬间会产生多次抖动，系统可能会把一次按键误判为多次按键。例如一次 `next` 操作可能导致歌曲连续切换多首，一次 `play/pause` 操作也可能导致播放状态反复翻转，最终表现为按键响应异常。

同时，按键输入相对于系统时钟 `clk` 是异步信号。如果直接送入时序电路，可能在触发器建立时间或保持时间附近变化，从而引起亚稳态。因此需要先通过同步器将异步按键信号同步到系统时钟域，再经过防颤动和脉宽变换电路，得到稳定的单周期按键脉冲，供 `mcu` 使用。

而 `reset` 按键通常用于系统初始化。即使复位按键存在抖动，其作用仍然是让系统保持或重新进入初始状态，多次复位一般不会造成逻辑功能错误。只要复位释放后系统能够进入稳定工作状态即可。因此在简单实验中，`reset` 通常可以不做与普通功能按键完全相同的防颤动处理。当然，在更严格的工程设计中，复位释放过程也常常需要同步化，以避免不同触发器在不同时间退出复位而产生不确定状态。

2. `mcu` 是否可能接收不到按键信息？若存在，概率为多大？是否需要修改？

答：在本实验设计中，`play/pause` 和 `next` 按键首先经过按键处理模块，输出已经同步到系统时钟域的单周期脉冲信号，然后再送入 `mcu`。由于按键处理模块和 `mcu` 使用同一个系统时钟，因此从同步时序电路的角度看，`mcu` 理论上可以在下一个时钟边沿正确采样到该脉冲，不应出现漏接按键信息的情况。也就是说，在时序约束满足、逻辑设计正确的前提下，漏接概率可以认为近似为 0。

如果不经同步化和脉宽变换，而是直接把外部异步按键信号送入 `mcu`，则确实可能出现接收不到或误接收的情况。原因包括：按键信号变化时刻与时钟采样边沿过近，可能引起亚稳态；或者有效脉冲宽度小于一个时钟周期，导致 `mcu` 没有在脉冲有效期间采样到它。此时漏接概率与按键信号宽度、系统时钟周期、触发器建立保持时间以及亚稳态恢复时间有关，不能简单给出固定值。

因此，对于本实验当前设计，不需要再对 **mcu** 作额外修改。按键处理模块已经完成了同步化、防颤动和单周期脉冲生成，能够保证 **mcu** 稳定接收按键信息。若在更复杂系统中存在跨时钟域传输，则需要增加握手电路、脉冲展宽电路或异步 FIFO 等方法进一步提高可靠性。

实验文件清单

类别	文件名
顶层文件与约束文件	MusicPlayerTop.v、music_player.v、MusicPlayer.xdc
音频接口相关文件	AudioInterface.v、AudioInterface.edf、AudioInterface.edfPreview
按键处理模块	button_unit.v、button_process_unit.v、button_process_unit.edf、debounce_filter.v、sync_ff.v
DDS 正弦波发生模块	dds.v、adder_22.v、addr_counter.v、addr_process.v、data_process.v、sine_rom.v、frequency_rom.v、dff_en_rst.v
同步与分频模块	sync_pulse.v、divider.v
主控制器模块	mcu.v
乐曲读取模块	song_reader.v、song_rom.v、song_counter.v、song_end_detector.v
音符播放模块	note_player.v、note_player_controller.v、note_reg.v、note_timer.v
仿真测试文件	button_process_unit_tb.v、dds_tb.v、divider_tb.v、mcu_tb.v、song_reader_tb.v、note_player_tb.v、music_player_tb.v、sync_pulse_tb.v

表 5: 实验工程文件清单